

Laboratorio No. 02 — OS Setup, Shell y Software de Soporte de Red

Por:
Camilo Aguirre
Juan David Rangel

Profesor: John Alexander Pachón Pinzón

Escuela Colombiana de Ingeniería Julio Garavito
Universidad

Arquitectura y Servicios de Red (AYSR)

Bogotá, D.C., viernes 21 de agosto de 2026

Índice general

0.1.	Introducción	2
0.2.	Marco teórico	2
0.2.1.	Packet Tracer	2
0.2.2.	Encapsulación	2
0.2.3.	Wireshark	2
0.2.4.	SMB / SAMBA	2
0.2.5.	Scripts de shell y editor VI	2
0.3.	Desarrollo	2
0.3.1.	1. Getting to Know Packet Tracer	2
0.3.2.	2. Tracking Messages with Packet Tracer	4
0.3.3.	3. Using Wireshark	5
0.3.4.	4. Network Cards	7
0.3.5.	5. Shell Programming — Unix (Slackware 15.0)	8
0.3.6.	6. Editor VI — Ejercicio del Himno	10
0.3.7.	7. Virtual Machine Deployment	12
0.3.8.	8. File Sharing — Servidor SMB/SAMBA en Solaris	12
0.4.	Resultados y conclusiones	15
A.	Anexo A — Enlaces a los videos	16
B.	Anexo B — Guía de sustentación (resumen)	17
B.1.	VI — conceptos clave	17
B.2.	SAMBA / SMB — conceptos clave	18

0.1. Introducción

Objetivo del laboratorio: familiarizarse con el software de soporte de red — Packet Tracer y Wireshark —, con la programación de scripts de shell, con el editor VI y con el despliegue de máquinas virtuales y compartición de archivos con SMB/SAMBA, sobre la plataforma de sistemas operativos montada en el Laboratorio 01.

Este informe es autosuficiente: toda la evidencia (capturas, comandos y salidas reales de ejecución) está incluida dentro del propio documento. Los videos de evidencia están publicados en YouTube y se enlazan en la sección correspondiente y en el Anexo A.

0.2. Marco teórico

0.2.1. Packet Tracer

Packet Tracer es un simulador de redes de Cisco que permite armar topologías, configurar dispositivos y observar el tráfico paquete por paquete en modo simulación. El curso introductorio de la plataforma (“Getting Started with Cisco Packet Tracer”) presenta los dispositivos de red, los modos de operación y el análisis de PDUs.

0.2.2. Encapsulación

Cada capa del modelo OSI/TCP-IP agrega su encabezado a los datos de la capa superior: los datos de aplicación se encapsulan en un mensaje ICMP/TCP, este en un paquete IP y este en una trama Ethernet. En el destino se desencapsula en orden inverso. Esto se observa tanto en Packet Tracer (PDUs) como en Wireshark (tráfico real).

0.2.3. Wireshark

Wireshark es un analizador de protocolos (sniffer) que captura los paquetes de una interfaz de red y los decodifica capa por capa. En modo promiscuo, la NIC captura todas las tramas del segmento, no solo las dirigidas a su propia MAC.

0.2.4. SMB / SAMBA

SMB (Server Message Block) es el protocolo de compartición de archivos e impresoras en redes Windows. SAMBA es la implementación libre que permite a sistemas Unix actuar como servidores o clientes SMB. Solaris 11.4 incluye un servidor SMB nativo gestionado por SMF y el paquete `service/network/samba` para el servicio clásico.

0.2.5. Scripts de shell y editor VI

El shell de Unix permite automatizar tareas de administración mediante scripts. VI es el editor de texto estándar de los sistemas Unix, con modos de operación (normal/inserción/comando).

0.3. Desarrollo

0.3.1. 1. Getting to Know Packet Tracer

Versión de Packet Tracer en la plataforma Cisco: 9.0.1 (disponible en NetAcad).

Video resumen del curso (capítulos 1–4, máx. 5 min): participan los dos integrantes. Duración: 3:42.

- Enlace: <https://youtu.be/RGuyi0b2aH4>

Quiz “Introduction to Packet Tracer - PT Basics Quiz” (individual): cada integrante rindió el quiz y guardó captura del resultado (figuras 1 y 2).

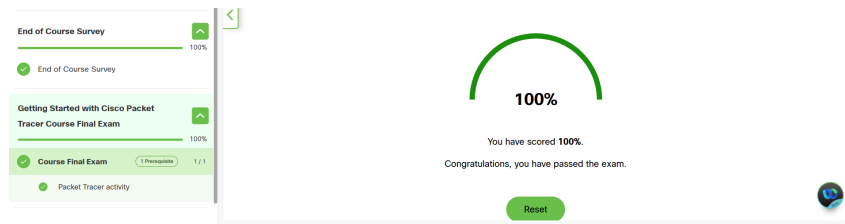


Figura 1: Resultado del quiz PT Basics (Juan David).

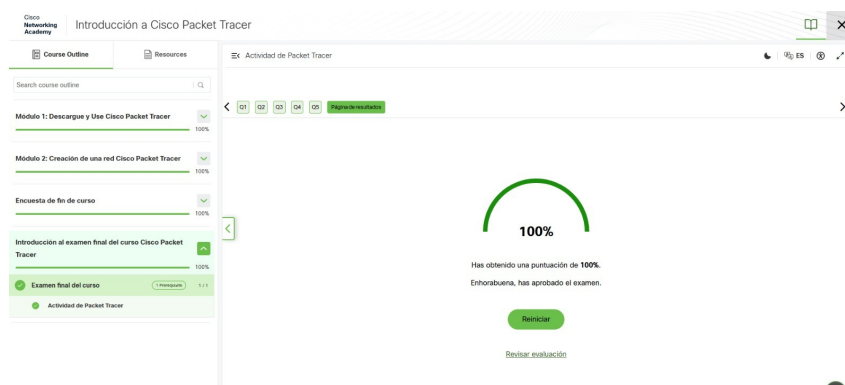


Figura 2: Resultado del quiz Getting Started (Camilo).

Diagrama de red en Packet Tracer (individual): topología con 3 routers (Router0 1941, Router1 1841, Router2 1841), switches y servidores, con enlaces Ethernet y un enlace serial entre routers (figura 3).

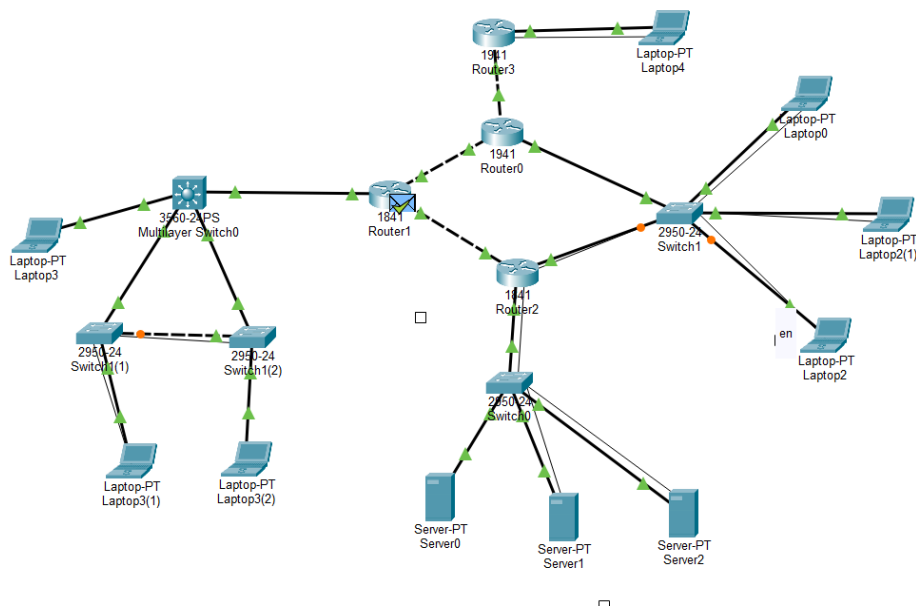


Figura 3: Topología de red armada en Packet Tracer.

Esquema de direccionamiento aplicado:

- WAN R0–R1: 7.0.0.0/30 (R0=.1, R1=.2)
- WAN R1–R2: 8.0.0.0/30 (R1=.1, R2=.2)
- LAN Router0 VLAN10: 10.0.10.0/24 (gw 10.0.10.1; Server0 .10, Server1 .11, Switch1 .2)
- LAN Router0 VLAN30: 10.0.30.0/24 (gw 10.0.30.1)
- LAN Router2: 10.0.20.0/24 (gw 10.0.20.1; Server2 .10)

Respuestas conceptuales:

- **Conexiones negras sólidas:** enlaces físicos Ethernet directos (cable de cobre) entre dispositivos; la conexión de capa física real que transporta tramas (L1/L2).
- **Conexiones negras punteadas:** conexiones lógicas o virtuales; no hay un cable físico directo, representan una relación lógica entre dispositivos (ruta a través de la red, enlace inalámbrico o un enlace que aún no está activo).
- **Cable serial Router0–Router2:** cable Serial DCE/DTE (rojo) entre los puertos seriales de los routers; el extremo DCE define el clock rate.

0.3.2. 2. Tracking Messages with Packet Tracer

Simulación: ping desde RADIUS hacia DHCP. Con la topología armada y las IPs configuradas, se activa el modo *Simulation* (abajo a la derecha), se filtran los protocolos **ICMP** y **ARP** en *Edit Filters*, y desde la consola del servidor RADIUS se ejecuta un **ping** a la IP del servidor DHCP.

Secuencia de eventos observada:

1. **ARP Request** (broadcast): RADIUS pregunta “¿quién tiene la IP del DHCP? Dame tu MAC”.
2. **ARP Reply:** el DHCP responde con su MAC.
3. **ICMP Echo Request:** mensaje de ping hacia el DHCP.
4. **ICMP Echo Reply:** respuesta del DHCP.

Si RADIUS y DHCP están en subredes distintas, entre el ARP y el ICMP aparecen los saltos del router: el paquete viaja con la MAC del siguiente salto en cada enlace, pero la IP origen/destino no cambia (enrutamiento de capa 3).

Análisis de las PDUs capa por capa (ICMP Echo Request):

- **Capa 2 (Ethernet):** MAC origen del RADIUS, MAC destino del DHCP (o del gateway), EtherType 0x0800 (IPv4).
- **Capa 3 (IP):** IP origen (RADIUS), IP destino (DHCP), protocolo 1 (ICMP), TTL 128 (decrementa en cada salto de router).
- **Capa 4 (ICMP):** Type 8 (Echo Request), Code 0, con la secuencia del ping.

El echo reply es el camino inverso: MACs e IPs intercambiadas y Type 0 (Echo Reply). El ARP request usa MAC destino FF:FF:FF:FF:FF:FF (broadcast) con EtherType 0x0806.

Conclusión: se observa la encapsulación/dencapsulación capa por capa: los datos del ping se encapsulan en un paquete IP y este en una trama Ethernet; en el destino se desencapsula en orden inverso. La IP es la dirección lógica de extremo a extremo (no cambia); la MAC es la dirección física salto a salto (cambia en cada enlace).

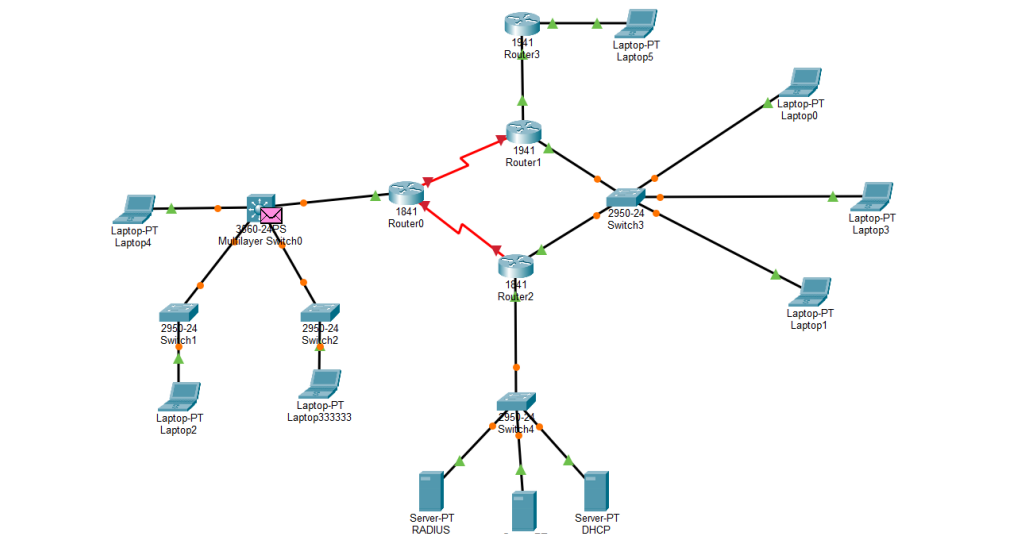


Figura 4: Trabajo de simulación en Packet Tracer.

0.3.3. 3. Using Wireshark

¿Qué es Wireshark? Analizador de protocolos de red (sniffer). Captura paquetes en vivo desde una interfaz, los decodifica capa por capa (Ethernet, IP, TCP/UDP, HTTP, DNS, etc.) y los muestra en tres paneles: lista de paquetes, detalle del paquete y bytes. Es multiplataforma.

¿Qué es el modo promiscuo? Normalmente la NIC solo procesa las tramas dirigidas a su propia MAC (o broadcast/multicast). En modo promiscuo captura TODAS las tramas del medio físico aunque no vayan dirigidas a ella; Wireshark lo usa para ver todo el tráfico del segmento.

Captura realizada (16/08/2026, Wireshark 4.6.7):

- Sitio consultado: <http://www.scielo.org.co> (HTTP en claro, para ver el GET)
- Duración: 30 segundos — Total: 1691 paquetes (1.558.566 bytes)
- Peticiones GET capturadas: 4, todas hacia 168.176.28.57 (servidor de SciELO)

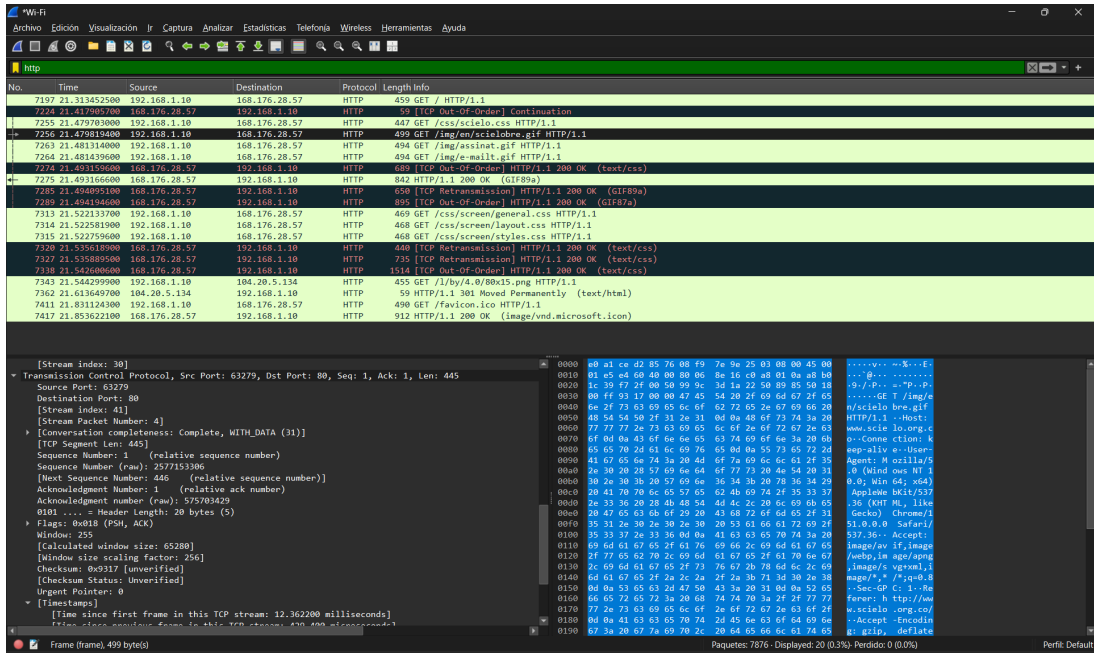


Figura 5: Captura en Wireshark del tráfico HTTP hacia scielo.org.co.

Análisis del paquete GET (frame #211, 216 bytes) — encapsulación capa por capa:

Capa	Protocolo	Datos del paquete
Enlace	Frame Ethernet II	216 bytes en el cable
Enlace	Ethernet II	MAC destino <code>e0:a1:ce:d2:85:76</code> (router) · MAC origen <code>08:f9:7e:9e:25:3b</code>
Red	IPv4	Origen <code>192.168.1.10</code> → destino <code>168.176.28.57</code> · TTL 128
Transporte	TCP	Puerto origen <code>60427</code> (efímero) → puerto <code>80</code> (HTTP) · flags PSH+ACK
Aplicación	HTTP	GET / HTTP/1.1 — petición de la página principal

Tabla 1: Desglose del paquete GET analizado.

Observación clave (encapsulación): cada capa agrega su encabezado a los datos de la capa superior. La petición HTTP va envuelta en TCP (capa 4), luego en IP (capa 3) y luego en Ethernet (capa 2). Al llegar al servidor se desencapsula en orden inverso.

Protocolos observados en la captura:

Protocolo	Paquetes	Nota
QUIC (UDP 443)	1266	Tráfico HTTPS moderno de fondo
TCP	168	Incluye la conexión HTTP a SciELO
HTTP	17	Las 4 peticiones GET + respuestas a SciELO
TLS	41	Handshakes HTTPS
DNS	9	Resolución de nombres (IPv4 + IPv6)
mDNS	4	Descubrimiento local
ICMPv6	4	Vecindario IPv6

Tabla 2: Resumen de protocolos de la captura.

Campo	Valor
Adaptador Wi-Fi	Realtek 8852CE WiFi 6E PCI-E NIC
MAC Wi-Fi	08-F9-7E-9E-25-03
IPv4	192.168.1.10/24 (DHCP)
Gateway / DNS	192.168.1.1
SSID / enlace	EVANGELIO_5G (5 GHz) — 866.7 Mbps
Bytes RX / TX	3.250.114.444 / 3.535.449.942
Adaptador Ethernet	Realtek PCIe GbE Family Controller (sin cable)

Tabla 3: Ficha de la tarjeta de red del host.

Otros adaptadores virtuales del host: Tailscale Tunnel, VirtualBox Host-Only (192.168.56.1) y 2 adaptadores Wi-Fi Direct.

Máquinas virtuales (2 comparadas con el host):

Campo	Slackware 15.0	Solaris 11.4
NIC	VirtualBox e1000	e1000g0 (Intel PRO/1000)
MAC	08:00:27:0a:46:19	virtual (dladm show-phys)
IPv4 (casa)	192.168.1.13 (DHCP)	192.168.1.11 (DHCP)
Velocidad	virtual	1000 Mb/s full-duplex

Tabla 4: Datos de red de las VMs comparadas.

Comparación host vs VMs: el host usa NICs físicas Realtek (Wi-Fi 6E + GbE); las VMs usan NICs virtuales emuladas. Las MAC de las VMs inician con 08:00:27 (OUI de VirtualBox), lo que demuestra que son virtuales. La velocidad del enlace Solaris es 1000 Mbps full-duplex (virtual), equivalente a la GbE física del host.

0.3.5. 5. Shell Programming — Unix (Slackware 15.0)

Se escribieron y ejecutaron los 4 scripts pedidos en Slackware 15.0 (guardados en /root/scripts/simples/). Se presentan en su versión más simple posible, con la explicación de cada línea.

Script 1.1 — mi_ls.sh (menú de listado)

```
#!/bin/bash
# Muestra un directorio de 3 formas distintas

read -p "Que directorio? " dir

echo "1) Normal    2) Con detalle    3) Con ocultos"
read -p "Opcion: " op

case $op in
  1) ls "$dir" ;;
  2) ls -l "$dir" ;;
  3) ls -a "$dir" ;;
esac
```

Bitácora línea por línea: `read -p` pregunta y guarda la respuesta en la variable `dir`; el menú guarda la elección en `op`; `case` elige qué ejecutar: `ls` simple, `ls -l` (con detalle: permisos, dueño, tamaño, fecha) o `ls -a` (incluye ocultos). Cada caso termina con `;;` y el menú cierra con `esac`.

Prueba real (opción 1 sobre /etc):

```
1) Normal    2) Con detalle    3) Con ocultos
DIR_COLORS
HOSTNAME
ImageMagick-7
ModemManager
NetworkManager
UPower
X11
```

Script 1.2 — buscar.sh (búsqueda)

```
#!/bin/bash
# Busca un archivo por nombre o una palabra dentro de un archivo

echo "1) Buscar archivo por nombre    2) Buscar palabra en archivo"
read -p "Opcion: " op

if [ "$op" = "1" ]; then
    read -p "Nombre (o parte): " nombre
    find . -name "$nombre*" 2>/dev/null

elif [ "$op" = "2" ]; then
    read -p "En que archivo? " archivo
    read -p "Que palabra? " palabra
    grep -n "$palabra" "$archivo"

else
    echo "Opcion no valida"
fi
```

Bitácora: `if [ "$op" = "1" ]` compara la elección (los espacios dentro de los corchetes son obligatorios). Si eligió 1, `find . -name "$nombre"` busca archivos cuyo nombre CONTENGA lo escrito (los `*` son comodín) y `2>/dev/null` esconde errores de permisos. Si eligió 2 (`elif`), `grep -n "$palabra$archivo"` busca la palabra dentro del archivo mostrando el número de línea. `else` atiende cualquier otra opción y `fi` cierra el bloque.

Prueba real (buscar “dev” en `/etc/fstab`):

```
1:/dev/sda3    /          ext4  defaults  1  1
2:/dev/sda1    /boot      ext4  defaults  1  2
3:/dev/sda2    swap       swap  defaults  0  0
```

Script 1.3 — revisar_logs.sh (logs)

```
#!/bin/bash
# Muestra las ultimas 15 lineas de los 3 logs del sistema

tail -15 /var/log/syslog
echo "-----"
tail -15 /var/log/messages
echo "-----"
tail -15 /var/log/secure 2>/dev/null || echo "(no hay log secure)"
```

Bitácora: `tail` muestra el FINAL del archivo (lo más reciente) y `-15` son las últimas 15 líneas. Se hace con los tres logs del sistema. En el último, `2>/dev/null || echo` esconde el error y avisa si el log no existe, para que el script no se rompa.

Respuestas sobre logs:

1. **¿Qué son los archivos de log?** Registros donde el SO y los servicios guardan eventos (fecha, origen, mensaje). Sirven para diagnóstico, auditoría y seguridad.

2. **¿Qué tipos de logs hay en los SO instalados?** Slackware: `syslog`, `messages`, `secure` (+ `dmesg/kernel`). Solaris: `/var/adm/messages`. Windows: Visor de eventos (System, Security, Application).
3. **¿Qué es syslog y qué define el estándar?** Protocolo/estándar de logging (RFC 5424): define formato (facilidad + severidad), transporte (UDP 514) y el servicio; centraliza logs de distintos dispositivos.
4. **¿Los logs encontrados siguen el estándar?** Sí: Slackware/Solaris usan formato `syslog` (facilidad.severidad, timestamp, host, proceso[pid]: mensaje). Windows usa Event Log propio (no `syslog` nativo).

Script 1.4 — `newgroup.sh` y `newuser.sh` (usuarios y grupos)

```
#!/bin/bash
# Crea un grupo si todavia no existe

read -p "Nombre del grupo: " nombre

if getent group "$nombre" > /dev/null; then
    echo "Ese grupo ya existe"
else
    groupadd "$nombre"
    echo "Grupo $nombre creado"
fi
```

Bitácora (`newgroup`): `getent group` le pregunta al sistema si el grupo YA existe (su respuesta se esconde con `>/dev/null`: solo importa si acertó o no). Si existe, avisa; si no, `groupadd` lo crea.

```
#!/bin/bash
# Crea un usuario con su grupo y su carpeta home

read -p "Usuario: " usuario
read -p "Grupo: " grupo

groupadd "$grupo" 2>/dev/null

useradd -m -g "$grupo" "$usuario"

passwd "$usuario"

echo "Listo: $usuario en el grupo $grupo"
```

Bitácora (`newuser`): `groupadd` asegura que el grupo exista (el error por duplicado se esconde para no romper). `useradd -m -g` crea el usuario: `-m` le crea la carpeta home y `-g` lo asigna al grupo. `passwd` pide la contraseña de forma interactiva.

0.3.6. 6. Editor VI — Ejercicio del Himno

Archivo de trabajo: `himno.txt` (16 líneas, 4 estrofas):

```
1 Estudiante, maestro la conquista
2 Sera hacer con amor nuestra labor
3 Cultores de espiritu humanista
4 Unidad de intelecto y corazon.
5 Escuela de ingenio es nuestra casa
6 Libro abierto a nuestra universidad
7 Aqui perdura mientras todo pasa
8 Cimiento de la fe y la integridad.
9 Ofrecemos la mano al que tropieza
10 La hidalguia del dialogo al rival
```

```

11 Ofrecemos la duda y la certeza
12 Mediamos entre hierro y el cristal.
13 Escuela de ingenio es nuestra casa
14 Libro abierto a nuestra universidad
15 Aqui perdura mientras todo pasa
16 Cimiento de la fe y la integridad.

```

Operaciones aplicadas con sus resultados reales:

- `:1,4s/a/-/g` — todas las “a” de las líneas 1–4 reemplazadas por “-”: `1,4 = rango · s = substitute · g = global` (todas las de cada línea).
- `:%s/al/##/g` — “al” por “##” en TODO el archivo (reemplazó en 3 líneas); `% = todo el archivo`.
- `dw` — borrar palabra bajo el cursor (borró “Estudiante” de la línea 1). `d=delete, w=word`.
- `:13,16d` — borrar las líneas 13–16 de una (rango + delete).
- `u` — deshacer: el archivo volvió a su estado anterior (historial de cambios).
- `G + gUU` — ir a la última línea y convertirla a MAYÚSCULAS: línea 16 quedó “CIMIENTO DE LA FE Y LA INTEGRIDAD.”.
- `7G yy yy G p` — a la línea 7, copiar 2 líneas y pegarlas al final (líneas 16–17 nuevas).
- `/Escuela` — búsqueda: “Escuela” aparece en las líneas 5 y 13; `n` va a la siguiente coincidencia.
- `5G` — ir a la línea 5.
- `:wq` — guardar y salir.
- `:1,5d` — al reabrir: borrar las primeras 5 líneas.

Tabla resumen de comandos VI:

Modo	Comando	Qué hace
Normal	<code>i / a</code>	Insertar antes / después del cursor
Normal	<code>Esc</code>	Volver al modo normal
Normal	<code>x / dw</code>	Borrar carácter / palabra
Normal	<code>dd / 5dd</code>	Borrar línea / 5 líneas
Normal	<code>u / Ctrl+r</code>	Deshacer / rehacer
Normal	<code>yy / p</code>	Copiar línea / pegar
Normal	<code>G / 5G</code>	Ir al final / a la línea 5
Normal	<code>gUU</code>	Línea actual a MAYÚSCULAS
Normal	<code>/palabra</code>	Buscar (<code>n</code> siguiente)
Comando	<code>:w</code>	Guardar sin salir
Comando	<code>:q / :wq / :q!</code>	Salir / guardar y salir / salir sin guardar
Comando	<code>:%s/old/new/g</code>	Reemplazar en todo el archivo
Comando	<code>:1,4s/a/-/g</code>	Reemplazar en rango de líneas
Comando	<code>:13,16d</code>	Borrar rango de líneas
Comando	<code>:set number</code>	Mostrar números de línea

Tabla 5: Comandos VI aplicados y verificados en el ejercicio.

0.3.7. 7. Virtual Machine Deployment

Se clonaron las 3 máquinas del laboratorio (Slackware 15.0, Solaris 11.4 y Windows Server GUI) y los clones pasaron a ser las VMs principales del grupo, con **auto-detección de red al arranque**: si detectan el gateway de la universidad (10.2.65.1) aplican IPs estáticas 10.2.78.74/.75/.76; si detectan la red de la casa, usan DHCP. Cada VM conserva un snapshot de respaldo `estado-final-2026-08-17`.

Verificación de conectividad (21/08/2026, red de la casa):

Origen → Destino	Resultado	Detalle
Slackware (.13) → Solaris (.11)	Éxito	2/2, 0 % loss, rtt 1.6 ms
Slackware (.13) → Windows (.14)	Éxito	2/2, 0 % loss, rtt 2.0 ms
Slackware (.13) → Internet	Éxito	2/2, 0 % loss
Solaris (.11) → Slackware (.13)	Éxito	alive
Solaris (.11) → Windows (.14)	Éxito	alive
Solaris (.11) → Internet	Éxito	alive
Windows (.14) → Slackware (.13)	Éxito	2/2, 0 % loss
Windows (.14) → Solaris (.11)	Éxito	2/2, 0 % loss
Windows (.14) → Internet	Éxito	2/2, 0 % loss

Tabla 6: Matriz de pings entre las 3 VMs (todas las pruebas con 0 % de pérdida).

Salidas reales representativas:

```
# Slackware -> Solaris (ping -c 2 192.168.1.11)
PING 192.168.1.11 (192.168.1.11) 56(84) bytes of data.
64 bytes from 192.168.1.11: icmp_seq=1 ttl=255 time=1.88 ms
64 bytes from 192.168.1.11: icmp_seq=2 ttl=255 time=1.30 ms
--- 192.168.1.11 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1004ms

# Solaris -> Windows e Internet (ping <ip> 2)
192.168.1.14 is alive
8.8.8.8 is alive

# Windows -> Slackware (ping -n 2 192.168.1.13)
Reply from 192.168.1.13: bytes=32 time=2ms TTL=64
Reply from 192.168.1.13: bytes=32 time=1ms TTL=64
Packets: Sent = 2, Received = 2, Lost = 0 (0% loss)

# Windows -> Internet (ping -n 2 8.8.8.8)
Reply from 8.8.8.8: bytes=32 time=12ms TTL=119
Reply from 8.8.8.8: bytes=32 time=6ms TTL=119
Packets: Sent = 2, Received = 2, Lost = 0 (0% loss)
```

0.3.8. 8. File Sharing — Servidor SMB/SAMBA en Solaris

Servidor (Solaris 11.4): SAMBA clásico (paquete `service/network/samba 4.7.6`), compartiendo el recurso compartido en el puerto 445.

Configuración `/etc/samba/smb.conf`:

```
[global]
workgroup = WORKGROUP
server string = Servidor SAMBA Solaris (Lab 02)
security = user
passdb backend = tdbsam
map to guest = Bad User
```

```

server min protocol = SMB2
[compartido]
path = /export/compartido
writable = yes
valid users = claudia admin
create mask = 0660
directory mask = 0770

```

Comandos del servidor (con salidas):

```

# pkg list samba
# service/network/samba 4.7.6-11.4.0.0.1.14.0

# svcadm enable svc:/network/smb/server
# netstat -an | grep .445          # *.445 LISTEN

# smbpasswd -a claudia             # Added user claudia
# smbpasswd -a admin              # Added user admin
# pdbedit -L                      # claudia:100, admin:104

# svcadm disable svc:/network/smb/server # apagar nativo (fallaba passwords)
# /usr/sbin/smbd -D                # Samba clasico, escucha 445

```

Nota: el servidor SMB nativo de Solaris (svc:/network/smb/server) presentó problemas de sincronización de contraseñas SMB con usuarios locales (LOGON_FAILURE); la solución fue usar el Samba clásico del mismo paquete.

Cliente Slackware 15.0:

```

# smbclient -L //10.2.78.75 -U claudia%claudia123
#      Sharename      Type      Comment
#      compartido    Disk      Carpeta compartida Lab 02 (AYSR)

# smbclient //10.2.78.75/compartido -U claudia%claudia123 -m SMB2 \
#      -c 'put /tmp/mensaje.txt desde-slackware.txt; get desde-slackware.txt /tmp/vuelta.txt'
# putting file ... desde-slackware.txt OK
# getting file ... de vuelta OK (contenido identico)

```

Cliente Windows Server GUI:

```

# net use ||10.2.78.75\compartido /user:claudia claudia123
# The command completed successfully.

# dir ||10.2.78.75\compartido
#      local-test.txt, desde-slackware.txt, prueba-claudia.txt

# echo test > ||10.2.78.75\compartido\desde-windows.txt
# Creado OK

```

Interoperabilidad cruzada: Slackware crea desde-slackware.txt y Windows lo ve; Windows crea desde-windows.txt y Slackware lo ve con ls.

```
root@solaris:~# Aug 17 13:51:49 solaris sendmail[911]: unable to qualify my own
domain name (solaris) -- using short name
Aug 17 13:51:49 solaris sendmail[911]: [ID 702911 mail.alert] unable to qualify
my own domain name (solaris) -- using short name
█
```

Figura 8: Servidor SMB/SAMBA en Solaris.

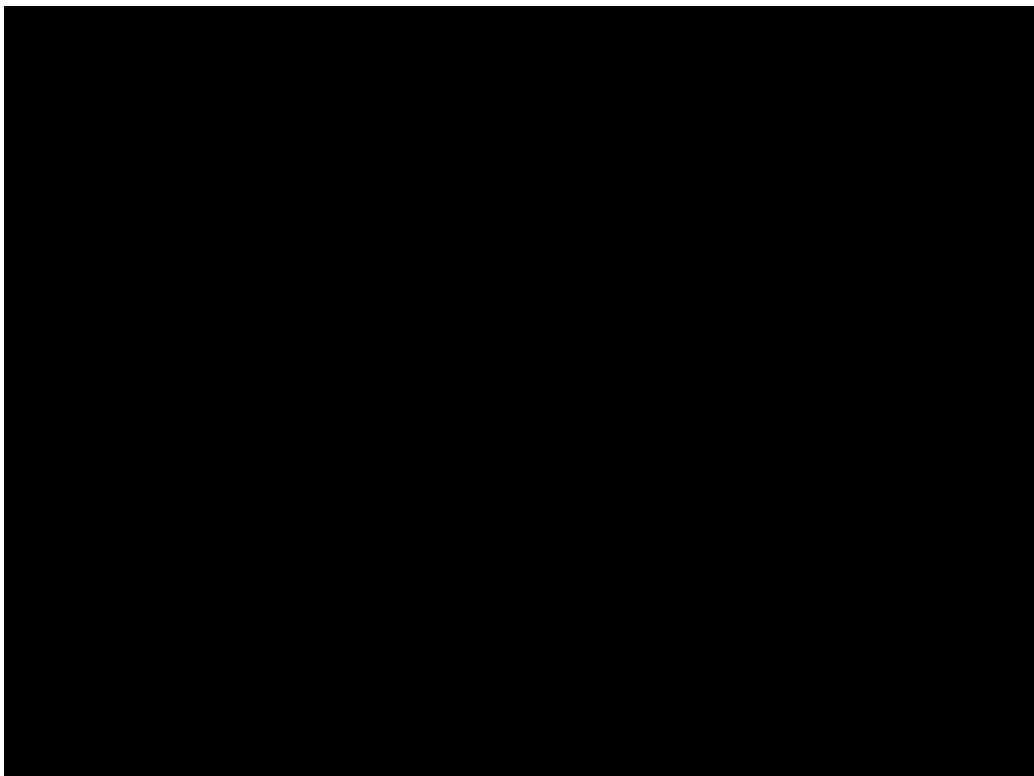


Figura 9: Cliente Windows accediendo al recurso SMB de Solaris.

0.4. Resultados y conclusiones

- Se completó el curso introductorio de Packet Tracer (versión 9.0.1) con video resumen y quiz individual.
- Se armó el diagrama de red en Packet Tracer con enlaces Ethernet y serial, y se analizó la simulación de mensajes (ping RADIUS-DHCP) con el desglose de PDUs capa por capa, confirmando el concepto de encapsulación.
- Con Wireshark se capturó y analizó tráfico real (scielo.org.co), contrastando la encapsulación observada en el simulador con la red real.
- Se documentaron las tarjetas de red del host y de las VMs, evidenciando la diferencia entre NICs físicas y virtuales.
- Se desarrollaron y probaron los 4 scripts de shell y el ejercicio completo del editor VI, con salidas reales verificadas.
- Las 3 VMs clonadas se comunican entre sí y tienen salida a Internet, con auto-detección de red entre la casa y la universidad.
- Se implementó compartición de archivos SMB/SAMBA entre Solaris (servidor), Slackware y Windows (clientes) con interoperabilidad cruzada verificada.

Apéndice A

Anexo A — Enlaces a los videos

- Video 1 — Curso Packet Tracer (3:42): <https://youtu.be/RGuyi0b2aH4>
- Video 2 — Wireshark interfaz y filtros (3:37): <https://youtu.be/n4p8lmUTW8w>
- Video 3 — Wireshark hallazgos (4:29): <https://youtu.be/ozmRhLV4BQs>

Apéndice B

Anexo B — Guía de sustentación (resumen)

B.1. VI — conceptos clave

VI tiene 3 modos: **Normal** (navegar, borrar, copiar — es el inicial), **Inserción** (escribir; se entra con **i** o **a** y se sale con **Esc**) y **Comando** (guardar, salir, buscar, reemplazar; se entra con **:**). Las teclas significan *verbos*, no letras que se escriben.

Desglose de los comandos más preguntados:

- **:1,4s/a/-/g**: 1,4 = rango de líneas · **s** = substitute · **/a/** = patrón · **-** = reemplazo · **g** = global (todas las de cada línea).
- **:%s/al/##/g:%** = todo el archivo (equivalente a **1,\$**); el resto igual.
- **dw**: **d** = delete (verbo) + **w** = word (objeto). VI combina verbos+objetos: **dd** borra línea, **5dd** borra 5 líneas.
- **u** / **Ctrl+r**: deshacer / rehacer (historial de cambios en memoria).
- **G** / **5G** / **7G**: ir a la última línea / línea 5 / línea 7.
- **gUU**: línea completa a mayúsculas (**guu** = minúsculas, **gUw** = palabra).
- **yy** + **p**: yank (copiar línea) + paste (pegar después; **P** pega antes).
- **/palabra**: buscar hacia adelante; **n** siguiente, **N** anterior.
- **:w** guardar sin salir · **:wq** guardar y salir · **:q!** salir descartando.
- **:set number**: mostrar números de línea.

Preguntas típicas de sustentación (VI):

1. ¿**Por qué no puedo escribir al abrir VI**? Porque se está en modo Normal; hay que pulsar **i** para entrar a modo Inserción.
2. ¿**Diferencia entre i y a**? **i** inserta antes del cursor, **a** después (append).
3. ¿**Qué significa la g en :1,4s/a/-/g**? Global: reemplaza todas las coincidencias de cada línea; sin **g** solo la primera.
4. ¿**Qué es % en :%s/al/##/g**? Todo el archivo.
5. ¿**Cómo deshacer**? **u** en modo normal.

6. ¿Cómo guardar sin salir / guardar y salir? :w y :wq.
7. ¿Cómo salir sin guardar? :q! (el ! fuerza y descarta).
8. ¿Qué hace 5G? Ir a la línea 5.
9. ¿Cómo copiar y pegar una línea? yy copia, p pega.
10. ¿Cómo buscar una palabra? /palabra, n para la siguiente.

B.2. SAMBA / SMB — conceptos clave

- **SMB**: protocolo de Windows para compartir archivos en red (Server Message Block).
- **SAMBA**: implementación libre de SMB para Unix/Linux/Solaris: hace que un sistema Unix “hable Windows”.
- **Share**: recurso compartido: la carpeta publicada para la red.
- **Puerto 445/TCP**: donde escucha SMB.
- **smbpasswd / tdbSAM**: base de usuarios SMB, separada de los usuarios del sistema.
- **security = user**: cada acceso exige usuario+contraseña.

Desglose de comandos clave (servidor):

- **ps -ef | grep smbd**: ps = process status · -e = todos los procesos · -f = formato completo · **grep smbd** = filtra el daemon de Samba.
- **netstat -an | grep .445**: -a todas las conexiones · -n numérico · puerto 445 = SMB; LISTEN = esperando clientes.
- **testparm**: valida la sintaxis de `/etc/samba/smb.conf` y muestra la configuración efectiva.
- **smbpasswd -a claudia**: agrega el usuario a la base SMB; la contraseña SMB es INDEPENDIENTE de la del sistema (-x elimina, sin opción cambia la contraseña).
- **pdbedit -L**: lista los usuarios SMB con su uid Unix.
- **/usr/sbin/smbd -D**: el daemon de Samba corriendo en segundo plano (-D = daemon).

Desglose de comandos clave (cliente):

- **smbclient -L //IP -U usuario**: -L enumera los shares del servidor.
- **smbclient //IP/compartido -U usuario -c 'put x; get y'**: -c ejecuta comandos sin entrar a la sesión interactiva (put sube, get baja).
- **net use**

IP\compartido /user:usuario contraseña: mapeo de unidad de red en Windows (ruta UNC).

- **smbstatus**: muestra las sesiones activas (quién está conectado, desde qué IP, con qué versión de SMB).

Parámetros de `smb.conf`:

- `workgroup = WORKGROUP`: grupo de trabajo estilo Windows al que se anuncia el servidor.
- `server string`: descripción que ven los clientes al enumerar.
- `passdb backend = tdbsam`: dónde se guardan las contraseñas SMB (base de datos local).
- `map to guest = Bad User`: un usuario inexistente NO se convierte en invitado (más seguro).
- `path = /export/compartido`: la carpeta REAL del disco que se comparte.
- `writable = yes`: se puede escribir (si `no` = solo lectura).
- `valid users = claudia admin`: lista blanca: SOLO estos usuarios pueden entrar.
- `create mask = 0660` / `directory mask = 0770`: permisos de archivos/carpetas nuevas (octal: dueño, grupo, otros).

Preguntas típicas de sustentación (SAMBA):

1. **¿Qué es Samba?** Implementación libre del protocolo SMB para que sistemas Unix compartan archivos con Windows.
2. **¿Qué es un share?** Una carpeta publicada en la red (path + permisos + usuarios).
3. **¿En qué puerto escucha SMB?** 445/TCP.
4. **¿Por qué `smbpasswd -a`?** Porque los usuarios SMB tienen su propia contraseña, separada de la del sistema.
5. **¿Qué significa `security = user`?** Autenticación obligatoria por usuario; sin credenciales no se accede.
6. **¿Qué hace `valid users`?** Restringe el share solo a esos usuarios.
7. **¿Para qué sirve `testparm`?** Validar la configuración antes de reiniciar el servicio.
8. **¿Cómo verifico quién está conectado?** `smbstatus`.
9. **¿Cómo conecto desde Windows?** `net use`

IP\share /user:usuario contraseña.
10. **¿Qué es un mask 0770?** El dueño y el grupo tienen permisos completos; los demás, ninguno.